Universidad de San Carlos de Guatemala
Escuela de Ingeniería en Ciencias y Sistemas, Facultad de Ingeniería
Introducción a la programación y computación 2, 2do. Semestre 2025

DESARROLLO DE UN SISTEMA DE FACTURACIÓN Y PAGOS BASADO EN API REST Y ALMACENAMIENTO XML PARA LA EMPRESA ITGSA

202408977 – Jonathan Eduardo Fuentes Garcia

Resumen

El presente trabajo describe el diseño e implementación de una solución de software para la empresa Industria Típica Guatemalteca S.A. (ITGSA), compuesta por un API REST desarrollada en .NET y una aplicación web frontend en ASP.NET Core MVC. El sistema permite el registro incremental de clientes y bancos a través de archivos XML de configuración, así como la recepción de transacciones de facturación y pagos. La lógica de negocio implementa la aplicación automática de pagos a las facturas más antiguas de cada cliente, generando saldos a favor cuando corresponde. Los datos se persisten en archivos XML utilizando serialización, y la comunicación entre el frontend y el API se realiza íntegramente en formato XML.
Los resultados muestran que el sistema cumple con los requerimientos funcionales exigidos, garantizando integridad en el procesamiento de pagos y generación de reportes de estado de cuenta e ingresos, con posibilidad de exportación a PDF.

Palabras clave
API REST, XML, facturación, pagos, .NET

Abstract

This paper describes the design and implementation of a software solution for the company Industria Típica Guatemalteca S.A. (ITGSA), consisting of a REST API developed in .NET and a frontend web application in ASP.NET Core MVC. The system allows the incremental registration of clients and banks through XML configuration files, as well as the reception of invoicing and payment transactions. The business logic implements automatic payment application to the oldest invoices of each client, generating credit balances when appropriate. Data is persisted in XML files using serialization, and communication between the frontend and the API is performed entirely in XML format. The results show that the system meets the required functional requirements, guaranteeing integrity in payment processing and generation of account statement and income reports, with PDF export capability.

Keywords
REST API, XML, invoicing, payments, .NET.

Introducción

Las empresas modernas requieren sistemas capaces de integrar diferentes canales de venta y pago de forma automatizada. ITGSA planteó la necesidad de un backend que centralizara las facturas generadas por su tienda virtual y los paogs provenientes de las bancas en línea, aplicando reglas contables específicas: los pagos deben abonarse a las facturas más antiguas, y cualquier exceso queda como saldo a favor del cliente.

Este proyecto aborda esa problemática mediante el desarrollo de un API REST en C# (.NET) y una interfaz web frontal que permite cargar archivos de configuración y transacciones en formato XML, consultar el estado de cuenta de los clientes, visualizar gráficamente los ingresos por mes y exportar reportes a PDF. El diseño se fundamenta en los principios de programación orientada a objetos (POO) y el uso de archivos XML tanto para la transferncia de datos como para la persistencia.

Desarrollo del tema

a. Arquitectura del sistema

El sistema se divide en dos programas principales: un **servicio** backend (API) y un cliente frontend (aplicación web Razor Pages). El frontend se comunica exclsivamente con el API mediante peticiones HTTP, intercambiando datos en formato XML tanto en las solicitudes como en las respuestas.

El API expone los siguientes endpoints funcionales:

- POST /api/Reinicio: elimina todos los datos.
- POST /api/Configuracion/cargar: recibe un archivo config.xml con clientes y bancos.
- POST /api/Transacciones/cargar: recibe un archivo transac.xml con facturas y pagos.
- GET /api/Consultas/estadoCuenta?nit=: devuelve el estado de cuenta de uno o todos las clientes.
- GET /api/Consultas/ingresos?mes=&anio=: devuelve los ingresos totales de los últimos tres meses.

b. Modelo de datos y persistencia en XML

El modelo de datos se compone de cuatro entidades principales: Cliente, Banco, Factura y Pago. Aunque la persistencia se realiza en archivos XML mediante serialización, la estructura puede representarse de manera abstracta con un diagrama entidad-relación (DER), donde las referencias entre entidades se implementan por igualdad de atributos (por ejemplo, Factura.NitCliente debe coincidir con el Nit de un cliente existente). La Figura 1 muestra este diagrama.

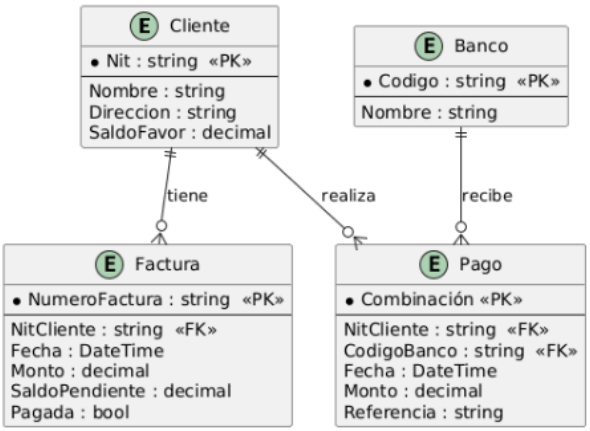

Figura 1. Diagrama entidad-relación del modelo de datos.

c. Lógica de aplicación de pagos

El núcleo funcional del sistema es la aplicación automática de pagos, implementada en la clase AplicadorPagos. El método AplicarPago recibe el NIT del cliente, el monto del pago y la lista global de facturas. Su flujo es el siguiente:

1. Seleccionar las facturas del cliente que no estén pagadas (Pagada ==false).
2. Ordenarlas por fecha ascendente (más antigua primero).
3. Recorrerlas en ese orden:
   - Si el monto restante del pago es mayor o igual que el saldo pendiente de la factura, se descuenta ese saldo del pago, la factura queda con SaldoPendiente = 0 y Pagada = true.
   - En caso contrario, se reduce el saldo pendiente de la factura y el monto restante del pago se vuelve cero.
4. Al final, el monto que no pudo aplicarse se devuelve como SaldoFavor dentro de un objeto ResultadoAplicacionPago, que también contiene la lista de facturas afectadas.

Este proceso garantiza que las facturas más antiguas se hagan primero, cumplendo con las reglas de negocio especificado en el enunciado del proyecto.

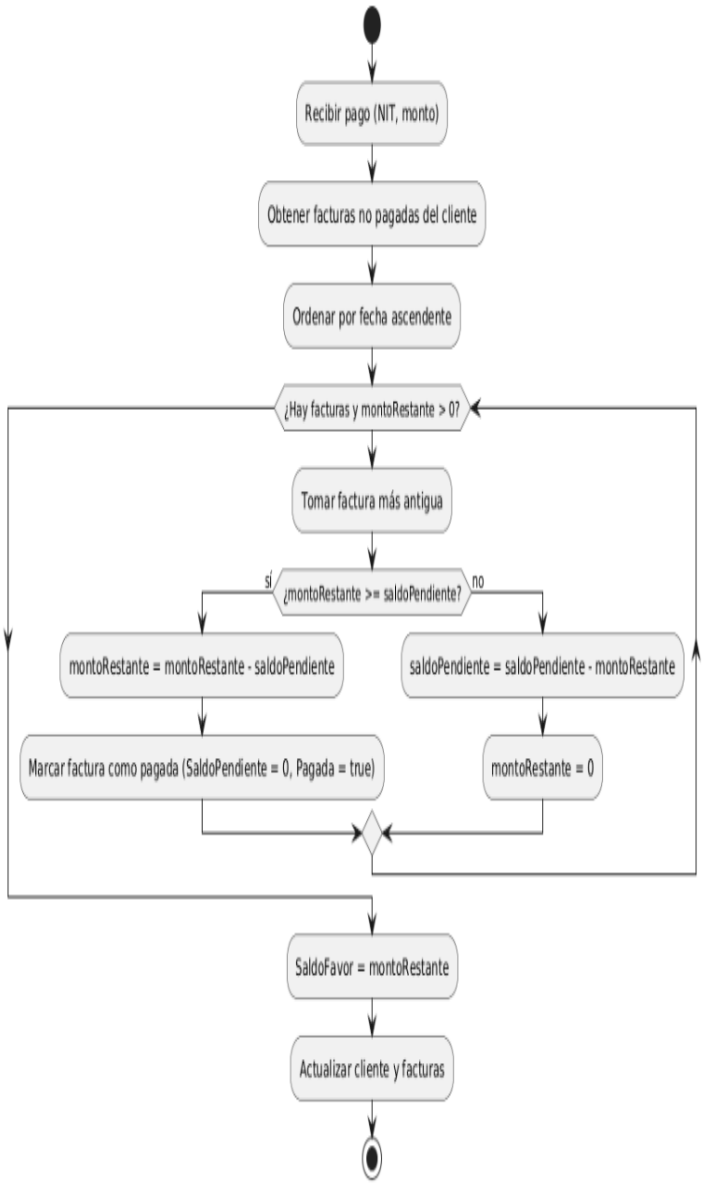

Figura 2. Diagrama de actividades del proceso de aplicación de un pago.

d. Diseño detallado de las clases y funcionalidades del frontend

La Figura 3 muestra el diagrama de clases completo del software, que incluye las entidades, la clase de almacenamiento y los servicios de aplicación de pagos.

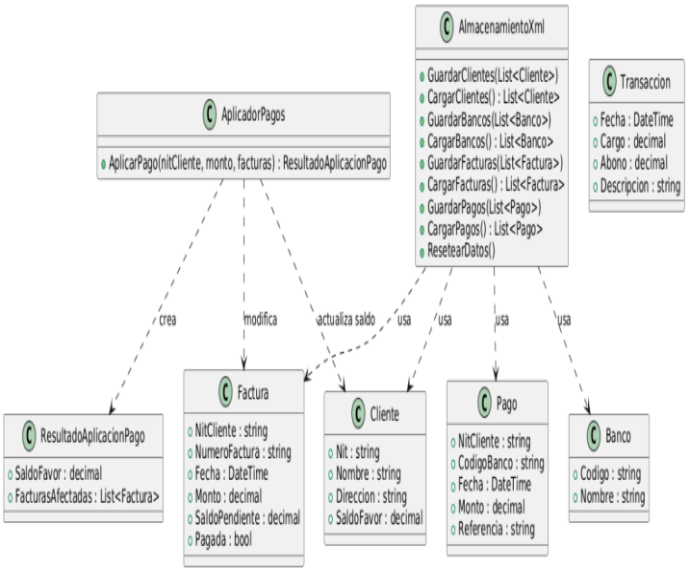

Figura 3. Diagrama de clases del sistema ITGSA.

La clase Transaccion es utilizada en el frontend y en el endpoint de estado de cuenta para construir la vista de transacciones, pero no se persiste directamente; se genera dinámicamente a partir de facturas y pagos.

En cuanto al frontend, la aplicación web ofrece las siguientes funcionalidades, todas ellas probadas y funcionales:

- **Reiniciar datos**: invoca el endpoint de reinicio y muestra un mensaje de confirmación.
- **Cargar Configuración**: formulario para subir config.xml y observar el XML de respuesta.
- **Cargar Transacciones**: formulario para transac.xml, que retorna las estadísticas de facturas/pagos.
- **Estado de Cuenta**: permite ingresar un NIT o dejar el campo vacío para obtener todos los clientes ordenados pr NIT. Muestra cada cliente en una tarjeta con su saldo actual y una tabla de transacciones (fecha, cargo, abono, descripción) ordenada de la más reciente a la más antigua. Incluye un botón que, utilizando las librerías jsPDF y html2canvas, exporta todo el contenido visible a un archivo PDF.
- **Ingresos**: solicita mes y año, y pinta una gráfica de barras con Chart.js mostrando los totales de ingresos de los últimos tres meses (incluyendo el solicitado). También puede exportarse la gráfica a PDF.
- **Ayuda**: muestra los datos del estudiante y una breve documentación del sistema.

Conclusiones

El sistema desarrollado para ITGSA satisface compltamente los requerimientos planteados: administra clientes y bancos mediante archivos XML incrementales, procesa facturas y pagos, aplica automáticamente los pagos a las facturas más antiguas, maneja saldos a favor y provee consultas de estado de cuenta e inresos. La comunicación en XML y la persistencia en archivos XML serializados garantizan la compatibilidad con el estándar solicitado.

La separación etre AP I y frontend, así como la implementación de controladores independientes para cada responsabilidad, permitió construir un software modular, fácil de probar con Postman y de extender si se requirieran nuevos servicios en el futuro.

Referencias bibliográficas

1. Microsoft. (2023). *ASP.NET Core MVC Overview*. Recuperado de https://learn.microsoft.com/en-us/aspnet/core/mvc/overview
2. W3C. (2008). *Extensible Markup Language (XML) 1.0 (Fifth Edition)*. Recuperado de https://www.w3.org/TR/xml/
3. Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. University of California, Irvine.
4. Date, C. J. (1991). *An introduction to Database Systems*. Addison-Wesley Publishing Company, Inc.
5. C. J. Date, (1991). *An introduction to Database Systems.* Addison-Wesley Publishing Company, Inc.